



23 | Sets and Tuples

Course: FastAPI - The Complete Course 2026 (Beginner + Advanced)

Lecture 23: Sets and Tuples (~7 min)

Sets

A set is similar to a list, but with two key differences: sets are **unordered** and **cannot contain duplicate values**. Sets use **curly brackets** `{}`.

```
my_set = {1, 2, 3, 4, 5, 1, 2}
print(my_set)      # {1, 2, 3, 4, 5}
print(len(my_set)) # 5
```

Even though 7 elements were defined, Python automatically removes the duplicates. Only the 5 unique values remain.

Looping Through a Set

```
for x in my_set:
    print(x)
```

Loops haven't been covered yet — they come in a later section.

No Indexing

Because sets are **unordered**, Python does not track element positions. Attempting to access by index raises an error.

```
print(my_set[0]) # ✗ TypeError
```

Modifying a Set

`discard(value)` — remove a specific element:

```
my_set.discard(3)
```

`clear()` — remove all elements:

```
my_set.clear()
```

`add(value)` — add a single element:

```
my_set.add(6)
```

`update(iterable)` — add multiple elements at once:

```
my_set.update([7, 8])
```

Tuples

Tuples are **ordered** like lists, but they are **unchangeable** (immutable). Tuples use **parentheses** `()`.

```
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple)          # (1, 2, 3, 4, 5)
print(len(my_tuple))     # 5
```

Indexing Works

Because tuples are ordered, you can access elements by index:

```
print(my_tuple[1])      # 2
```

No Modification Allowed

You **cannot** add, update, or reassign elements in a tuple:

```
my_tuple[0] = 100      # ❌ TypeError: 'tuple' object does not support item
                        assignment
```

Methods like `append()` or `insert()` simply don't exist on tuples.

Lists vs Sets vs Tuples

FEATURE	LIST <code>[]</code>	SET <code>{}</code>	TUPLE <code>()</code>
Ordered	Yes	No	Yes
Duplicates allowed	Yes	No	Yes
Mutable (changeable)	Yes	Yes	No
Indexable	Yes	No	Yes

When to Use Each

- **Sets** — when you need fast lookups or want to **remove duplicates** from a collection.
 - **Tuples** — when you want data to be **read-only** and protected from accidental changes.
 - **Lists** — the most commonly used of the three; suitable for most general-purpose tasks.
-

Key Takeaways

1. **Sets use** `{}`, are unordered, and automatically remove duplicates.
2. **You cannot index into a set** — there is no concept of position.
3. Use `discard()`, `add()`, `update()`, and `clear()` to modify sets.
4. **Tuples use** `()`, are ordered, and support indexing — just like lists.
5. **Tuples are immutable** — no adding, removing, or reassigning elements.
6. Lists remain the most frequently used data structure of the three.